Le modèle de rendement : entraînement et évaluation

DataFlow360 · Guide d'explication pour l'équipe · Partie ML

1. Le problème en une phrase

> On décrit une campagne agricole (culture, région, superficie, année, pluie…) et le modèle **prédit le rendement en tonnes par hectare**. Multiplié par la superficie, il donne la production.

C'est un problème de **régression** : on prédit un nombre, pas une catégorie.

La règle qui guide tout le projet : le modèle n'utilise que des informations qu'un utilisateur (une ONG ou le ministère) peut fournir. Deux colonnes disponibles dans les données sont donc volontairement exclues :

- **rendement_n_1** (rendement de l'année précédente) : il est inconnu après 2015, on ne pourrait pas le fournir en simulation.
- **production_t** : rendement = production / superficie. L'utiliser reviendrait à donner la réponse au modèle. C'est ce qu'on appelle une **fuite de données**.

2. Les données d'entrée

Source : la table Snowflake `ML.ML_RENDEMENT_CAMPAGNE`, construite par dbt. Une ligne correspond à une campagne, soit **9 894 campagnes**.

Variables catégorielles	Variables numériques
produit (10 cultures)	annee
region (14 régions)	superficie_ha
saison (Principale / Contre-saison)	pluie_cumul_mm (cumul sur les mois de la saison)
systeme_production (Pluvial…)	tavg_moyenne (température moyenne)

Cible : `cible_rendement_t_ha`, le rendement réel de la campagne.

Préparation des données (ml/scripts/donnees.py)

- Système de production vide → **Pluvial**, le cas par défaut au Sénégal.
- **Région sans station météo → climat de la région voisine** : Fatick et Kaffrine prennent celui de Kaolack, Sedhiou celui de Kolda. La même règle s'applique en simulation.
- **Les campagnes sans pluie connue sont écartées** (2 197) plutôt que d'inventer leur climat. Il reste **7 697 campagnes**.
- Les températures manquantes (aucune mesure avant 1973) sont remplacées par la médiane, calculée **uniquement sur les données d'entraînement**.

3. Le découpage des données : la clé de l'évaluation

Le découpage est **temporel**, pas aléatoire. C'est le point le plus important à retenir.

Apprentissage 1960 – 2005	Validation 2006 – 2010	Test 2011 – 2015
TRAIN : 6 456 campagnes		TEST : 1 241 campagnes

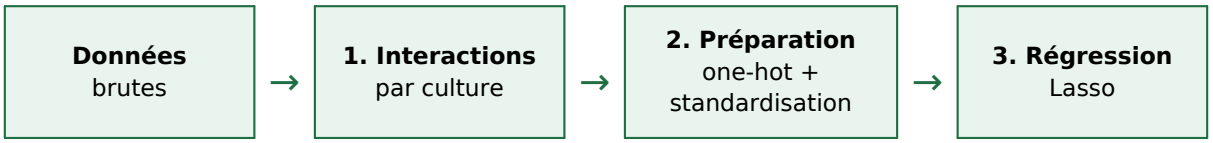
- **Pourquoi temporel ?** En vrai, on prédit le **futur** à partir du **passé**. Un découpage aléatoire laisserait le modèle « voir » 2014 pour prédire 2012, ce qui donnerait des scores trop optimistes.
- **Validation (2006-2010)** : sert à **choisir** le modèle et ses réglages.
- **Test (2011-2015)** : sert **uniquement à mesurer**, jamais à choisir. Sinon, on « optimiserait pour l'examen » et le score final serait biaisé.

> **Analogie** : le train, ce sont les cours ; la validation, les examens blancs qui servent à choisir sa méthode de révision ; le test, l'examen final qu'on ne passe qu'une fois.

Un test automatique (tests/test_ml/test_no_data_leakage.py) le vérifie : il multiplie par 100 les rendements du test et contrôle que le modèle choisi ne change pas.

4. Le modèle : un pipeline en 3 étapes

Toutes les transformations sont dans un pipeline scikit-learn : elles sont apprises sur le train, puis appliquées à l'identique en validation, en test et en production.



Étape 1 : les interactions par culture

Un modèle linéaire simple applique **le même effet de la pluie à toutes les cultures**. Or 100 mm de pluie en plus n'ont pas le même effet sur le mil que sur le riz irrigué. On ajoute donc :

- **région × culture** : le niveau habituel de rendement (le mil à Kaolack ne rend pas comme le mil à Louga) ;
- **pluie × culture** : une pente de pluie propre à chaque culture ;
- **année × culture** : une tendance dans le temps propre à chaque culture (semences, techniques…).

> Sans ces interactions, **aucun des modèles ne bat la baseline** (MAE test ≈ 0,67). C'est l'élément qui fait la différence.

Étape 2 : la préparation

- **One-hot encoding** des catégories : « Mil » devient une colonne qui vaut 0 ou 1.
- **Standardisation** des nombres (moyenne 0, écart-type 1). Sans elle, la pénalité de Ridge et Lasso traiterait différemment *annee* (≈ 2000) et *superficie_ha* (≈ 1 000).

Étape 3 : la régression

Modèle	Principe
Régression linéaire	Cherche la droite (l'hyperplan) qui colle le mieux aux données, sans contrainte.
Ridge	Linéaire + pénalité qui **réduit** les coefficients, pour éviter le sur-apprentissage.
Lasso	Linéaire + pénalité qui peut **annuler** des coefficients : il fait aussi le tri des variables inutiles.

Le paramètre **alpha** règle la force de la pénalité : plus alpha est grand, plus le modèle est simple.

5. Le déroulé de l'entraînement (ml/scripts/entrainement.py)

1. **Préparer** les données (section 2).
2. **Choisir le modèle** : 9 combinaisons (1 linéaire, 4 Ridge, 4 Lasso). Chacune apprend sur 1960-2005 et est notée sur 2006-2010. La plus petite erreur (MAE) gagne.
3. **Réentraîner** le gagnant sur tout le train (1960-2010).
4. **Évaluer** sur le test (2011-2015) et comparer à la baseline.
5. **Modèle de production** : mêmes réglages, réentraîné sur **toutes** les années (1960-2015) pour profiter des données les plus récentes. Il est sauvegardé dans ml/models/modele_rendement.pkl.
6. **Écrire les métriques** dans ml/models/metriques.json.

Résultats de la sélection (validation 2006-2010)

Candidat	MAE validation (t/ha)
Régression linéaire	0,750
Ridge, alpha = 1 (meilleur Ridge)	0,614
Lasso, alpha = 0,001 (retenu)	**0,578**

6. L'évaluation

Les trois métriques

Métrique	Signification	Comment l'expliquer
MAE	Erreur absolue moyenne	« En moyenne, on se trompe de 0,6 t/ha. »
RMSE	Racine de l'erreur quadratique moyenne	Elle punit fortement les grosses erreurs.
R^2	Part de la variabilité expliquée (0 à 1)	« Le modèle explique 65 % des différences de rendement. »

La baseline : le point de comparaison

La baseline est une référence volontairement simple : **le rendement moyen historique pour la région et la culture**. Si le modèle ne fait pas mieux que cette moyenne, il ne sert à rien. C'est la première question à se poser.

Résultats sur le test (2011-2015, 1 241 campagnes)

	MAE (t/ha)	RMSE	R^2
Baseline (moyenne région × culture)	0,634	1,477	0,609
Lasso (alpha = 0,001)	**0,608**	**1,387**	**0,655**

> Le modèle **bat la baseline** : MAE −4 %, RMSE −6 %, R^2 +0,05. Le gain est **modeste mais réel**.

Pourquoi le RMSE est-il plus de deux fois plus grand que la MAE ? Parce qu'il existe quelques très grosses erreurs. Elles viennent du **manioc** et de la **patate douce**, des tubercules aux rendements élevés (plusieurs t/ha) et très variables.

Détail par culture (MAE sur le test)

Culture	Campagnes	MAE modèle	MAE baseline	Verdict
Manioc	122	2,59	2,98	Meilleur
Riz	136	1,07	1,14	Meilleur
Niébé	176	0,13	0,14	Légèrement meilleur
Fonio	28	0,14	0,14	Légèrement meilleur
Sorgho	180	0,24	0,25	Légèrement meilleur
Arachide (en coque)	189	0,22	0,21	Équivalent
Mil	173	0,16	0,16	Équivalent
Maïs	178	0,52	0,41	Moins bon
Sésame	54	0,32	0,18	Moins bon
Patate douce	5	8,69	8,61	Non fiable (trop peu de données)

Pour situer : le mil rend environ 0,7 t/ha, avec une erreur de 0,16 t/ha, soit environ 20 %.

Un test de bon sens : la cohérence avec la pluie

Pour chaque culture pluviale et chaque région, on compare la prédiction avec 300 mm puis 600 mm de pluie. Dans **80 % des cas**, plus de pluie ne fait pas baisser le rendement prédit. Comme l'outil sert à simuler des scénarios « et si… », il doit réagir de façon plausible quand on change la pluie.

7. Les limites à annoncer franchement

- **L'effet de la pluie est une droite** : le modèle ne voit ni l'effondrement en cas de sécheresse, ni le plafond en année très humide. Il sous-estime aussi cet effet (mil à Kaolack en 2027 : 0,72 t/ha à 250 mm, 0,76 t/ha à 800 mm).
- **Années après 2015** : le modèle prolonge la tendance observée ; plus l'année est lointaine, moins c'est fiable.
- **Contre-saison** : données historiques uniquement à Saint-Louis ; ailleurs, la simulation est refusée.
- **Maïs et sésame** sont moins bien prédits que la simple moyenne ; la **patate douce** n'est pas fiable.

8. Questions fréquentes

Pourquoi pas un Random Forest ou XGBoost ?

Le modèle linéaire est interprétable, réagit de façon prévisible à la pluie et prolonge une tendance pour les années futures. Un modèle à base d'arbres ne sait pas extrapoler au-delà de 2015. C'est une bonne piste d'amélioration pour l'effet non linéaire de la pluie, mais l'extrapolation dans le temps resterait à gérer.

Pourquoi Lasso a-t-il gagné ?

Avec les interactions, il y a beaucoup de colonnes, dont beaucoup sont peu utiles. Lasso en met une partie à zéro, ce qui limite le sur-apprentissage.

Pourquoi réentraîner sur toutes les années pour la production ?

Une fois les réglages validés, on veut que le modèle profite des années les plus récentes (2011-2015). Les métriques publiées restent celles mesurées sur le test, avant ce réentraînement.

Pourquoi ne pas inventer la pluie des campagnes sans climat ?

La pluie est la variable centrale de la simulation. L'inventer ajouterait du bruit et masquerait la vraie relation entre pluie et rendement. On préfère moins de données mais des données fiables.